AN ILLUSTRATED GUIDE TO AI AGENTS · CHAPTER 1

What Is an AI Agent?

The foundations: from a reasoning LLM to memory, tools, planning and autonomy

Maarten Grootendorst & Jay Alammar · Chapter 1 summary

What this chapter covers

01

The definition

Environment, sensors, actuators and the agent program

02

The brain

LLMs and reasoning LLMs

03

Augmentation

Memory, tools, planning and reflection

04

The agentic system

Autonomy, use cases, responsible use and evaluation

05

Specializations

Multi-agent, multi-modal and coding agents

06

TinyAgent

The small skeleton agent built up chapter by chapter

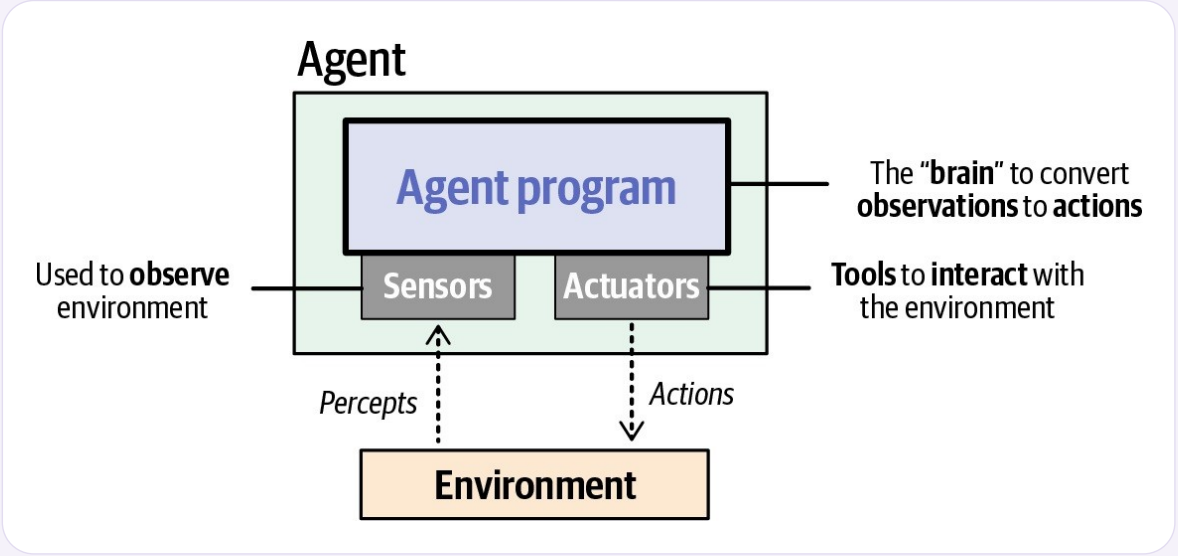
THE DEFINITION

“An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.”

Russell & Norvig, *Artificial Intelligence: A Modern Approach*

Broad enough to include non-LLM entities, structured enough to build on. This book covers LLM-backed agents.

Four components at the heart of every agent



Environment

The world the agent interacts with

Sensors

Components used to observe the environment

Actuators

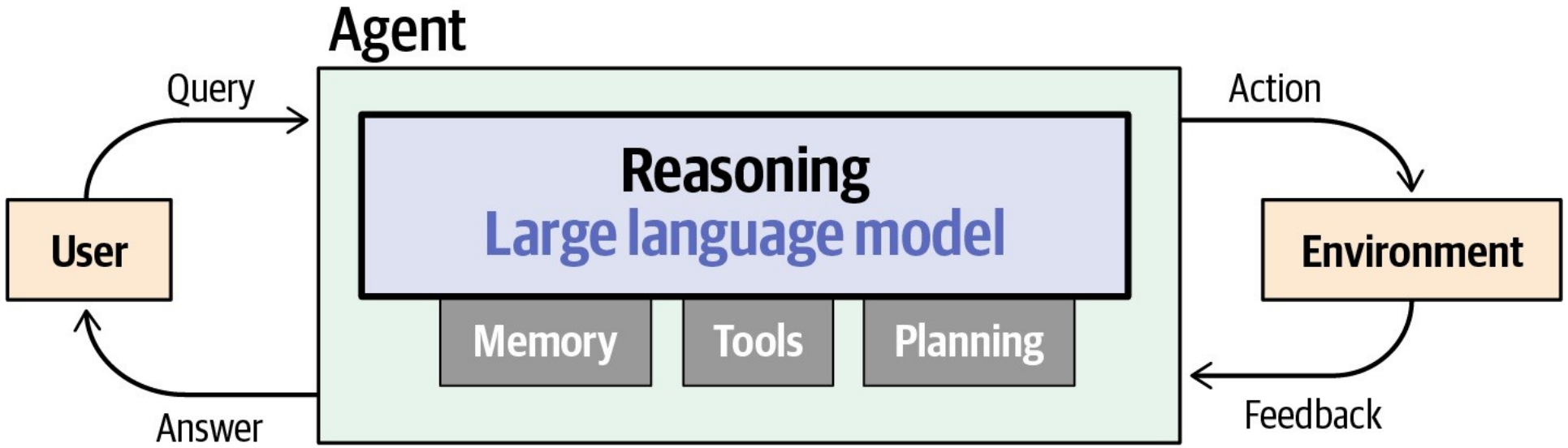
Tools the agent uses to act on the environment

Agent program

The "brain": rules for going from observations to actions

Figure 1-1 · An Illustrated Guide to AI Agents

Mapping the definition onto an LLM agent



Agent program = brain

A reasoning LLM extended with memory, tools and planning

Actuators = tools

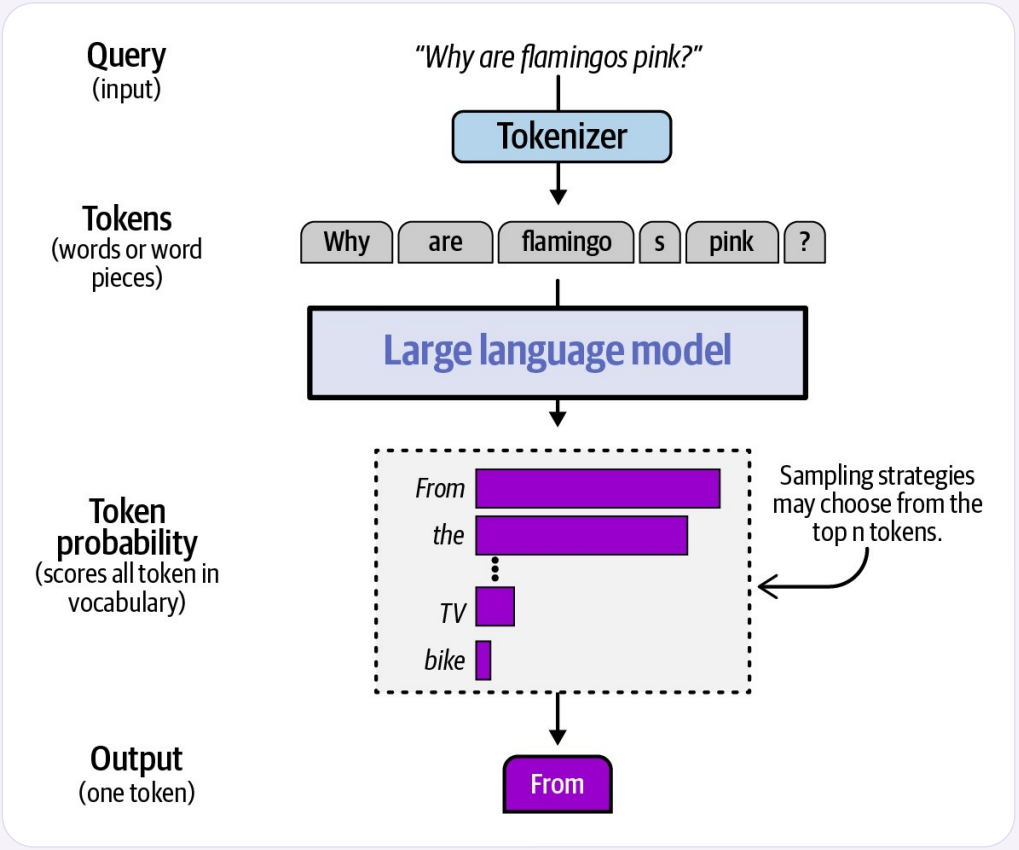
How the LLM acts on its environment

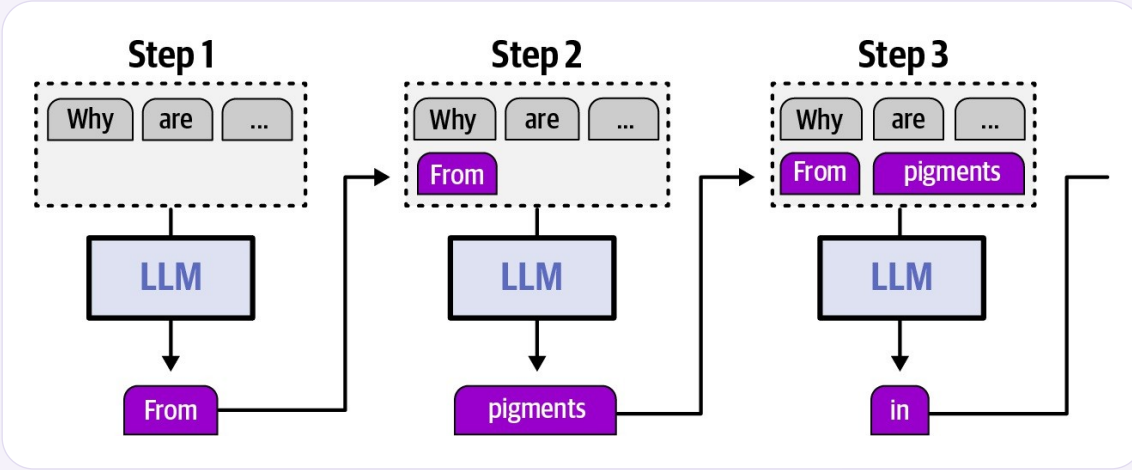
Sensors and user

Images or sound can act as sensors; the user is part of the environment

Figure 1-2 · An Illustrated Guide to AI Agents

LLMs predict one token at a time



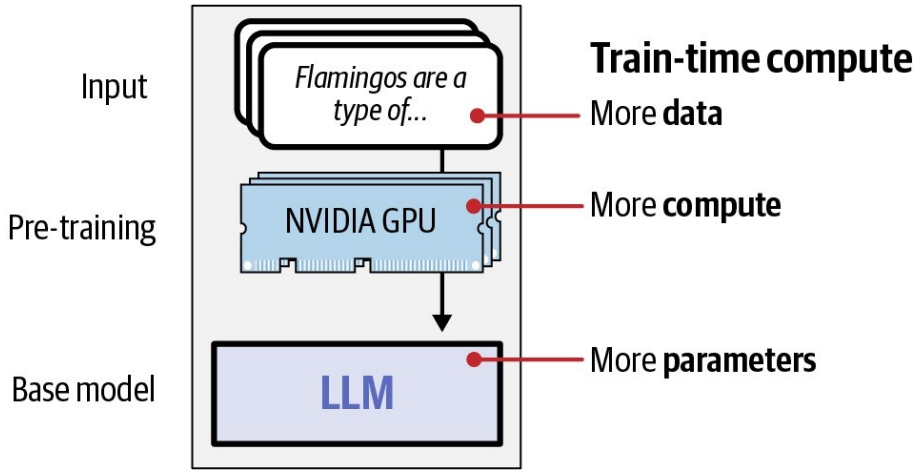


Tokens are sub-word pieces, so the model can generalize to words it has not seen.

Autoregression appends each prediction to the input and repeats, building a full answer.

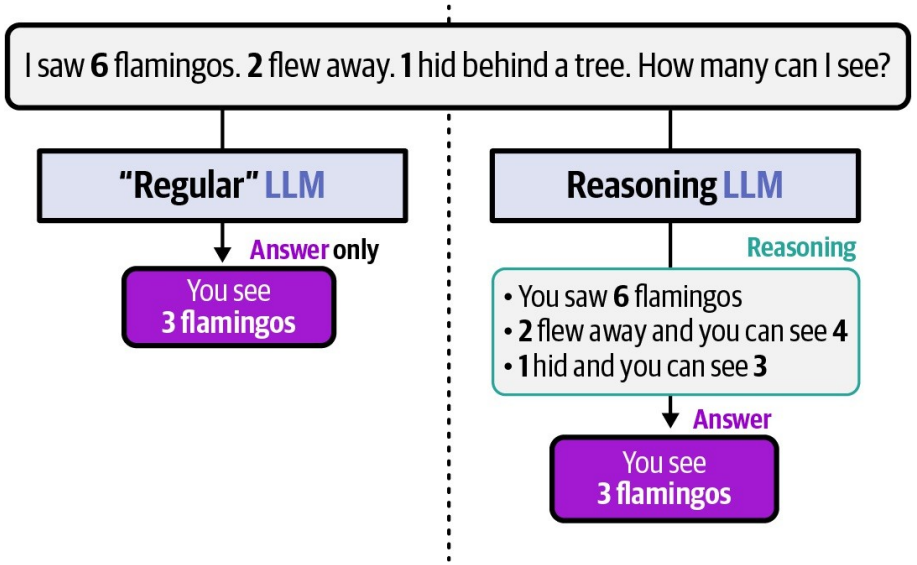
Figures 1-3 and 1-4 · An Illustrated Guide to AI Agents

From scaling training to scaling thinking



Train-time scaling

More data, compute and parameters gave predictable gains, until the cost outgrew the improvement.

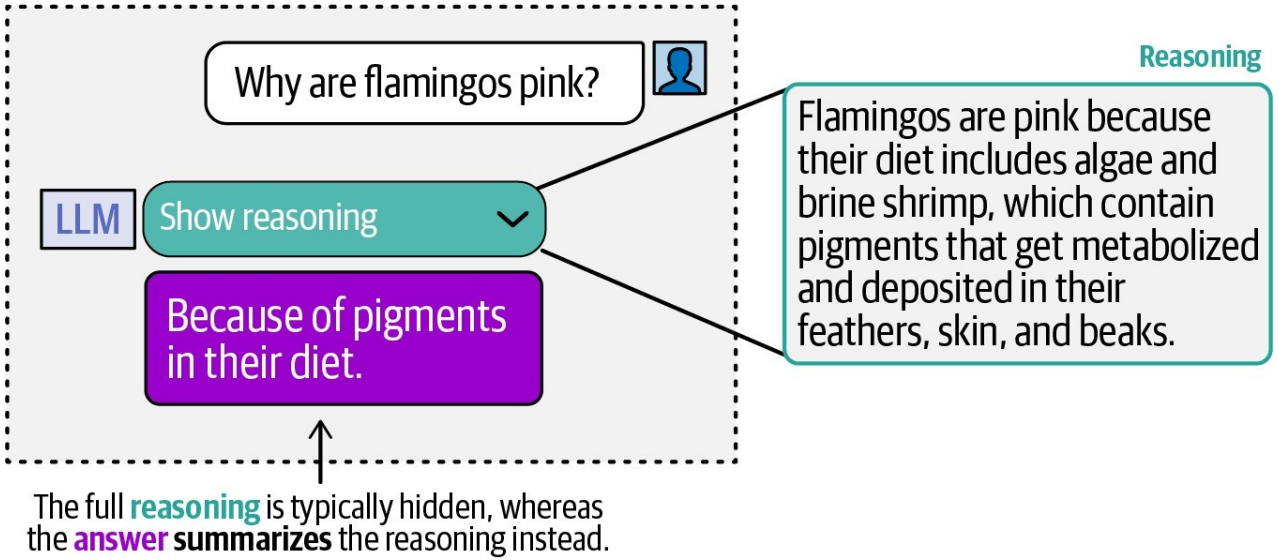


Reasoning LLMs

Models like OpenAI o1 and DeepSeek-R1 "think out loud", spending compute on reasoning tokens before the answer.

Figures 1-5 and 1-6 · An Illustrated Guide to AI Agents

Reasoning happens before the answer



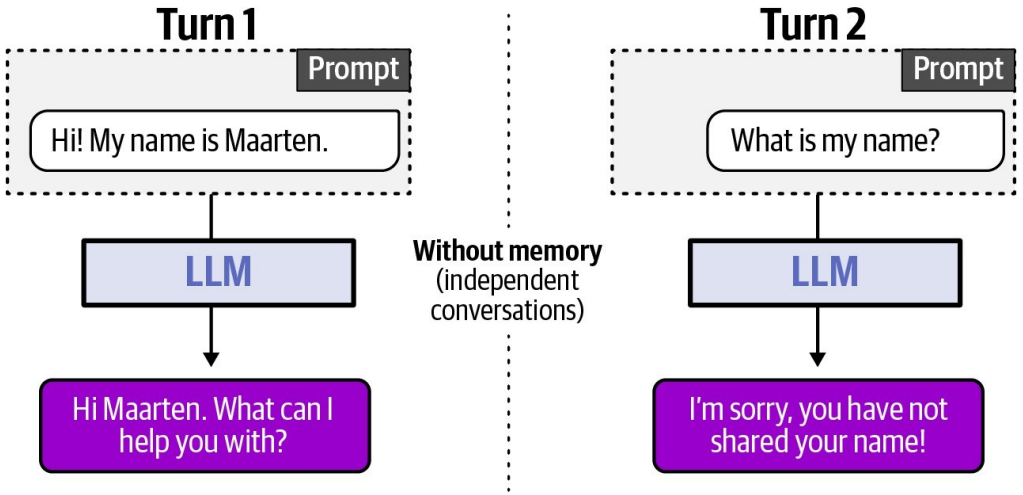
Think first. The LLM writes down its thoughts, then builds its answer on them.

Often hidden. Apps typically hide or summarize the reasoning trace.

Right tool for the job. Reasoning LLMs suit planning, tool choice and multi-step problems; regular LLMs suit fast, cheap replies.

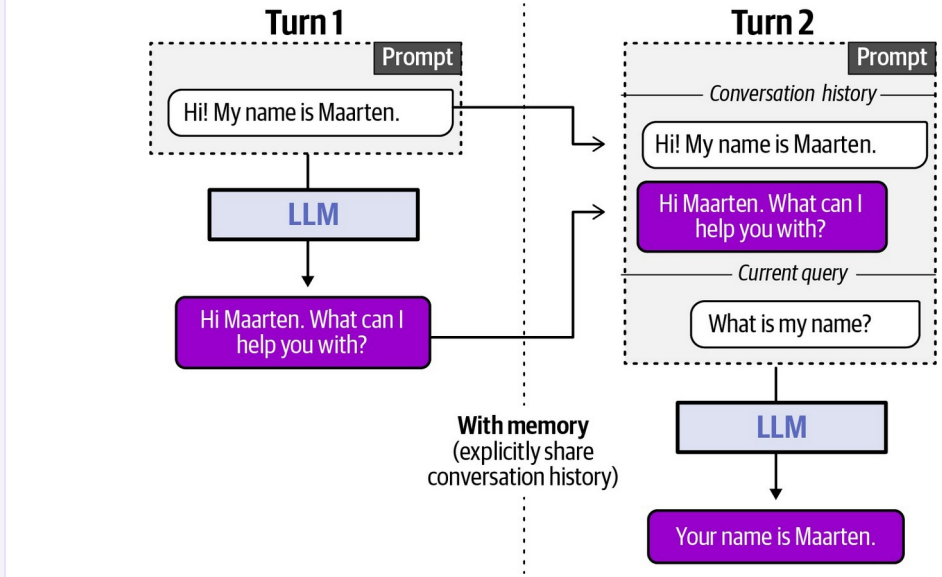
Figure 1-7 · An Illustrated Guide to AI Agents

Memory: LLMs are stateless by default



Without memory

Each call is independent, so the LLM is just an answering machine.

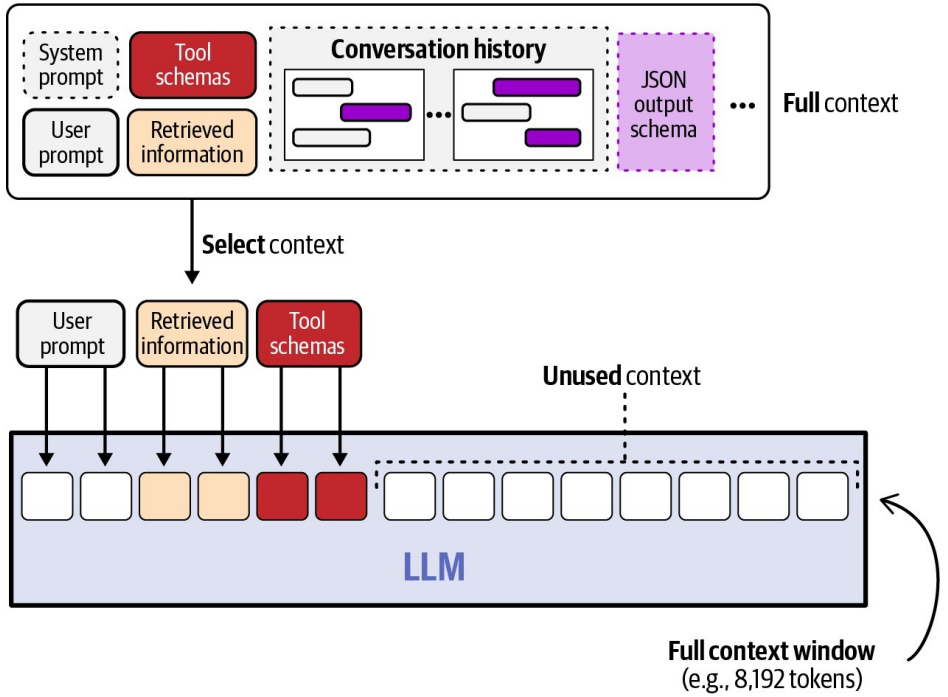


With memory

The simplest approach adds earlier turns to the current prompt.

Figures 1-9 and 1-10 · An Illustrated Guide to AI Agents

Context engineering: balance, not volume



Too much information causes **information overload**, even for LLMs, and can hurt decisions.

Context engineering balances the amount and quality of what we put in the prompt for the task at hand.

COMPETING FOR THE CONTEXT WINDOW

- System prompt
- User prompt
- Retrieved information
- Tool schemas
- Conversation history

Figure 1-11 · An Illustrated Guide to AI Agents

Tools: an LLM can only express intent



1 · Output

The LLM outputs text such as multiply(5.1, 7.3)

2 · Intent

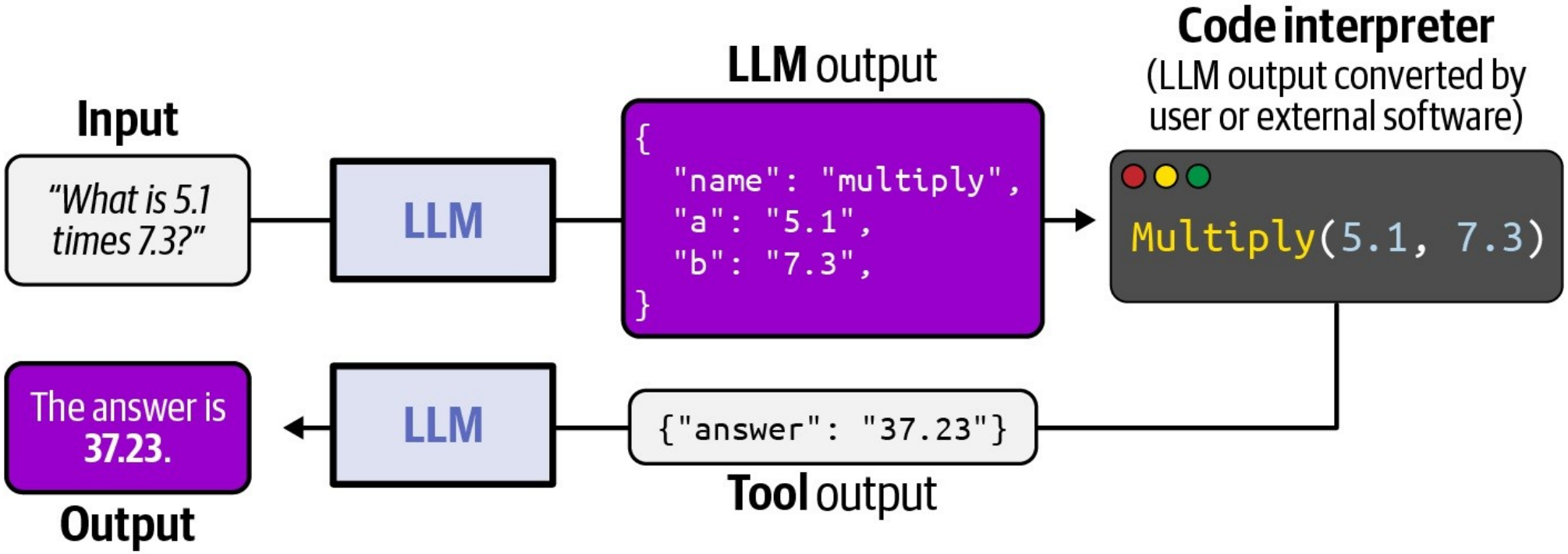
The string only shows intent; nothing runs without outside help

3 · Execution

Our software parses the text and turns it into a real tool call

Figure 1-12 · An Illustrated Guide to AI Agents

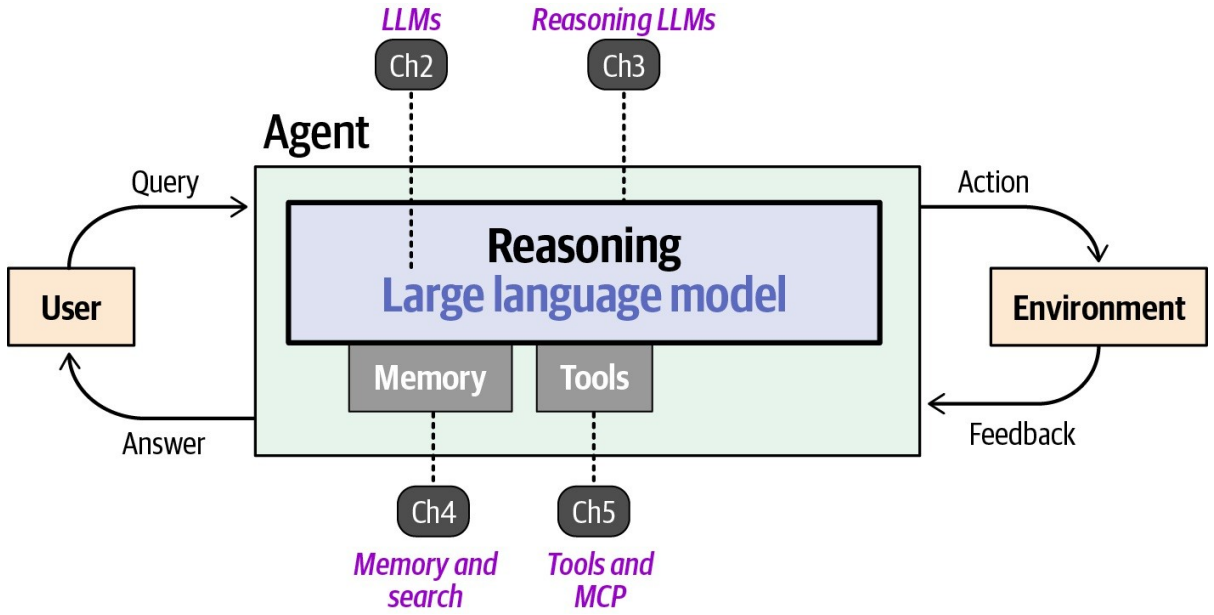
The tool-calling workflow



Chapter 5 covers how LLMs learn tools, and how the Model Context Protocol standardizes them across LLMs.

Figure 1-13 · An Illustrated Guide to AI Agents

The augmented LLM



A reasoning LLM with **memory** and **tools**, a term Anthropic uses, is the building block we turn into an agent.

Decides how to act

Which tools to use and how to use them

Decides what to keep

What information to retain from interactions

Figure 1-14 · An Illustrated Guide to AI Agents

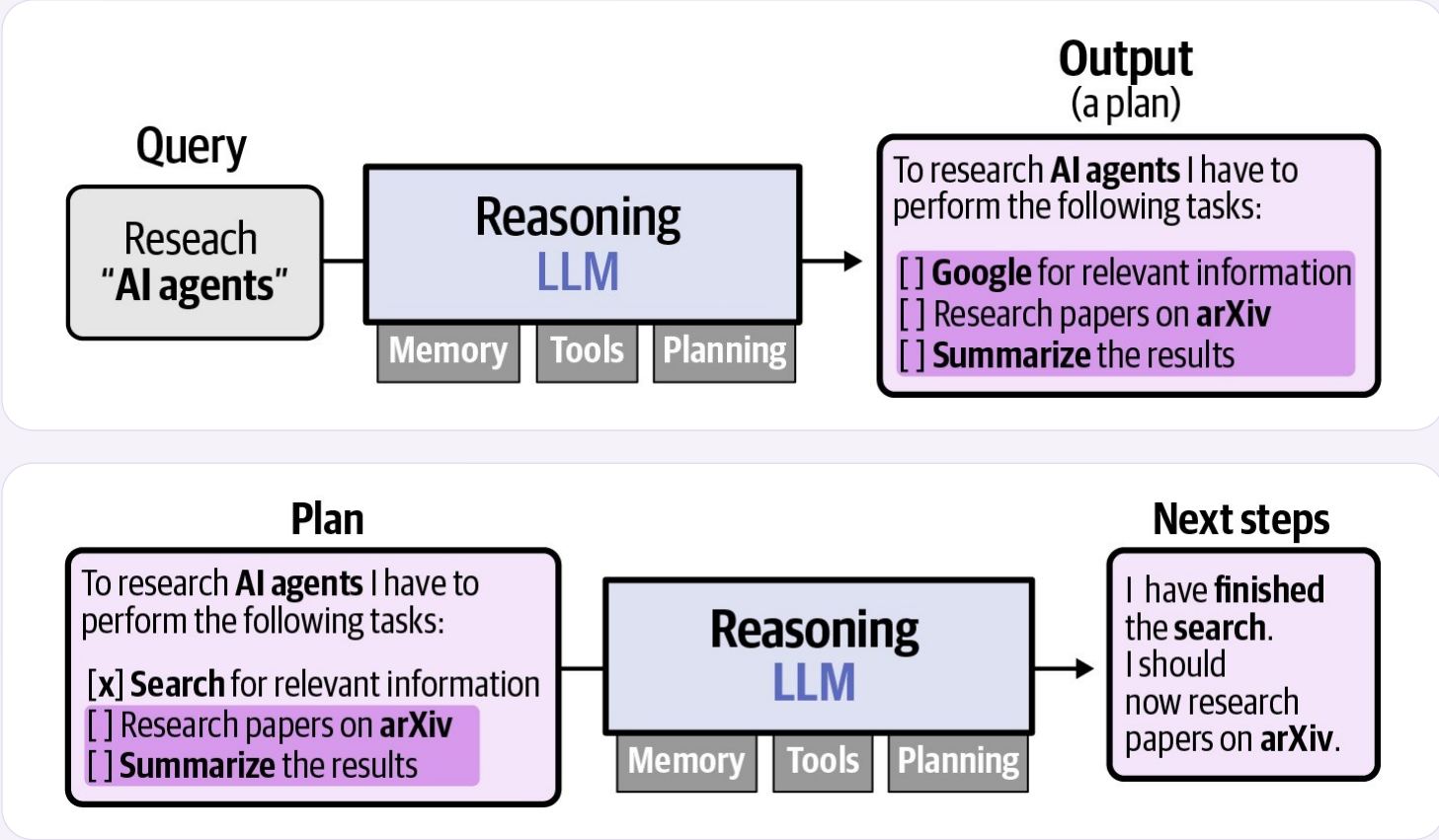
Planning: decompose, then execute

Task decomposition

Break a large task into smaller, actionable steps and track progress in a plan.

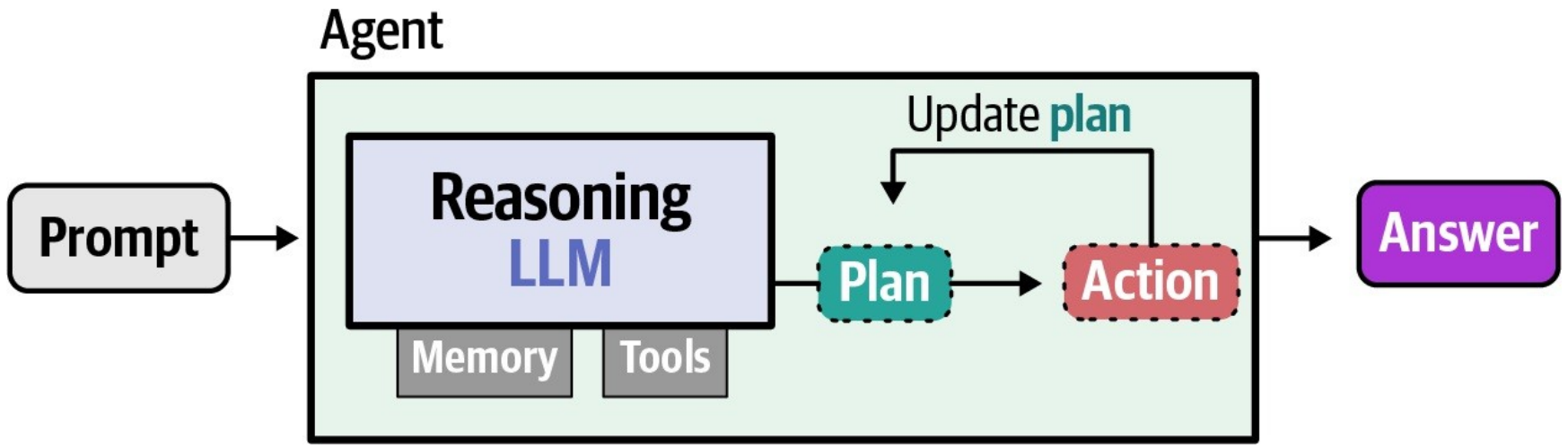
Reasoning picks the next step

After each task the agent reasons over the plan, for example marking Google search done and moving on to arXiv.



Figures 1-15 and 1-16 · An Illustrated Guide to AI Agents

Reflection: plans that improve themselves



Plan
Break the task into steps

Act
Take an action, such as a tool call

Reflect
Review the output, find faults, update the plan

Example: finding Google and arXiv insufficient, the agent adds Semantic Scholar and PubMed to its plan.

Figure 1-17 · An Illustrated Guide to AI Agents

The complete AI agent

Reasoning LLM + memory + tools + planning and reflection

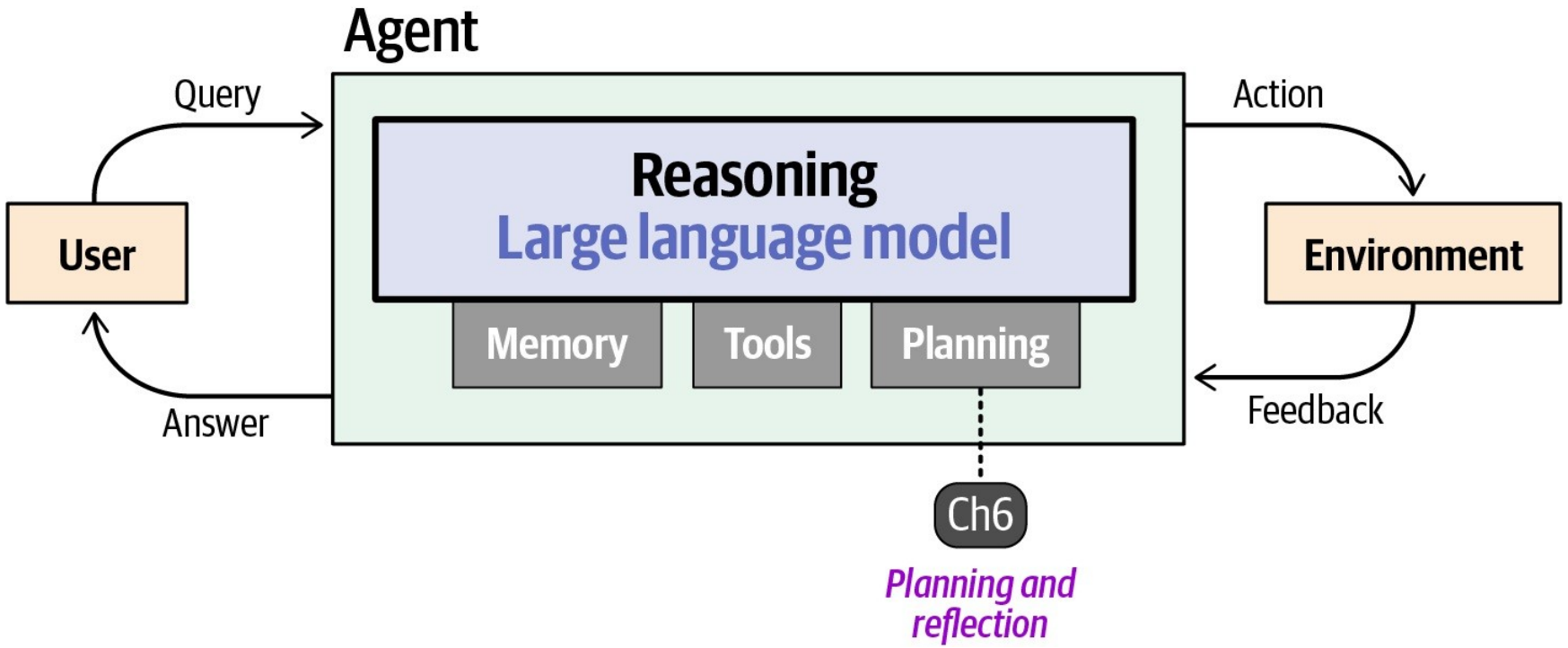
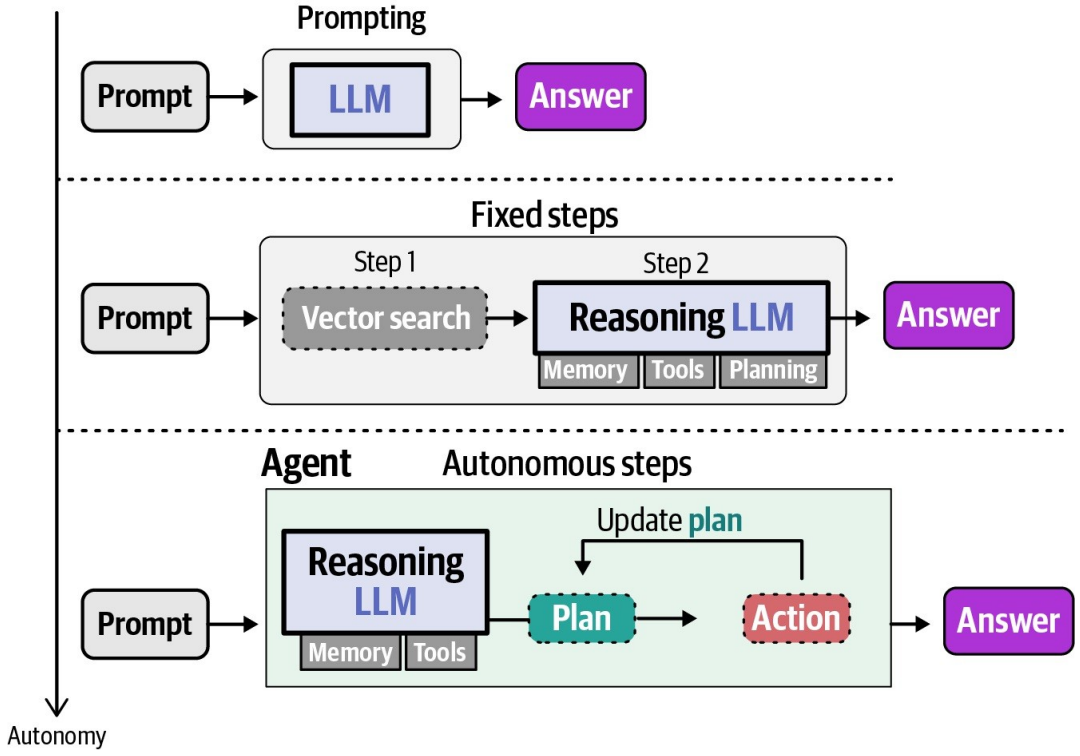


Figure 1-18 · An Illustrated Guide to AI Agents

Autonomy is a spectrum



Prompting

A single LLM generation

Fixed steps

An orchestrated pipeline

Autonomous steps

The agent chooses tools, updates its plan and decides when to stop

The authors' view

Goal-directed behavior and decisions make an agent, even with partial autonomy

Figure 1-19 · An Illustrated Guide to AI Agents

Where agents are most useful

Open-ended problems where the goal is clear but the path is not.

Coding

The most common use case: a clear goal, defined requirements, and results that can be verified automatically.

Deep research

Searches sources like arXiv, PubMed and Google Scholar with little user input: a great starting point on a new topic.

Automation

Standardizing processes, such as structuring patient data stored differently across hospitals.

Responsible agent development

Human in the loop

More autonomy needs humans to authorize, check and audit decisions, often reviewing output or intermediate steps.

Guardrails

Full autonomy can be overkill or harmful. Many guardrails steer the agent toward expected behavior.

Misinformation

Agents are still LLMs and can hallucinate, so critical systems need extra checks and balances.

It is all about context: an agent diagnosing patients with no human involvement is, with today's technology, harmful behavior.

Evaluating agents means evaluating a system

A single quality score for the final text rarely shows whether the agent did its job. Chapter 7 goes deeper.

MAIN LENS

Outcome

Did the task actually get done, such as the message sent or the record updated?

MAIN LENS

Trajectory

The steps and tool calls taken, judged on efficiency and soundness even when the outcome is correct.

ACROSS RUNS

Reliability

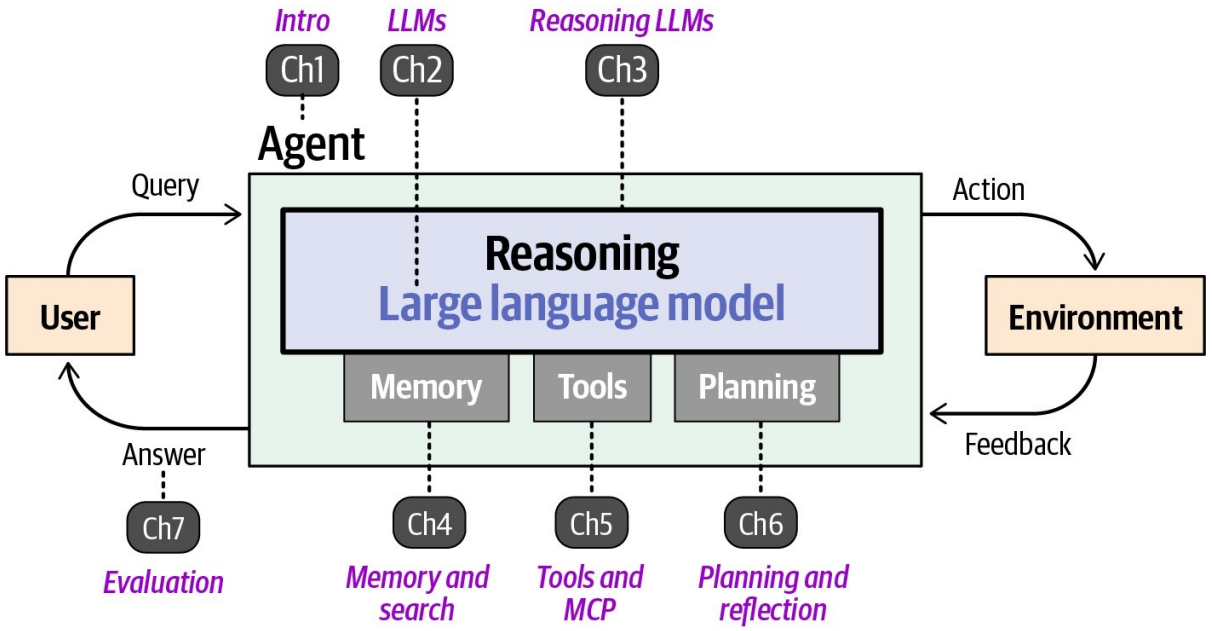
Does the agent succeed every time, not just once? Its outputs are stochastic.

ACROSS RUNS

Safety

Does it avoid harm from a malicious user, manipulated data it reads, or its own mistakes?

Part I: a single agent, built piece by piece



Ch 2 Large language models

Ch 3 Reasoning LLMs

Ch 4 Memory and search

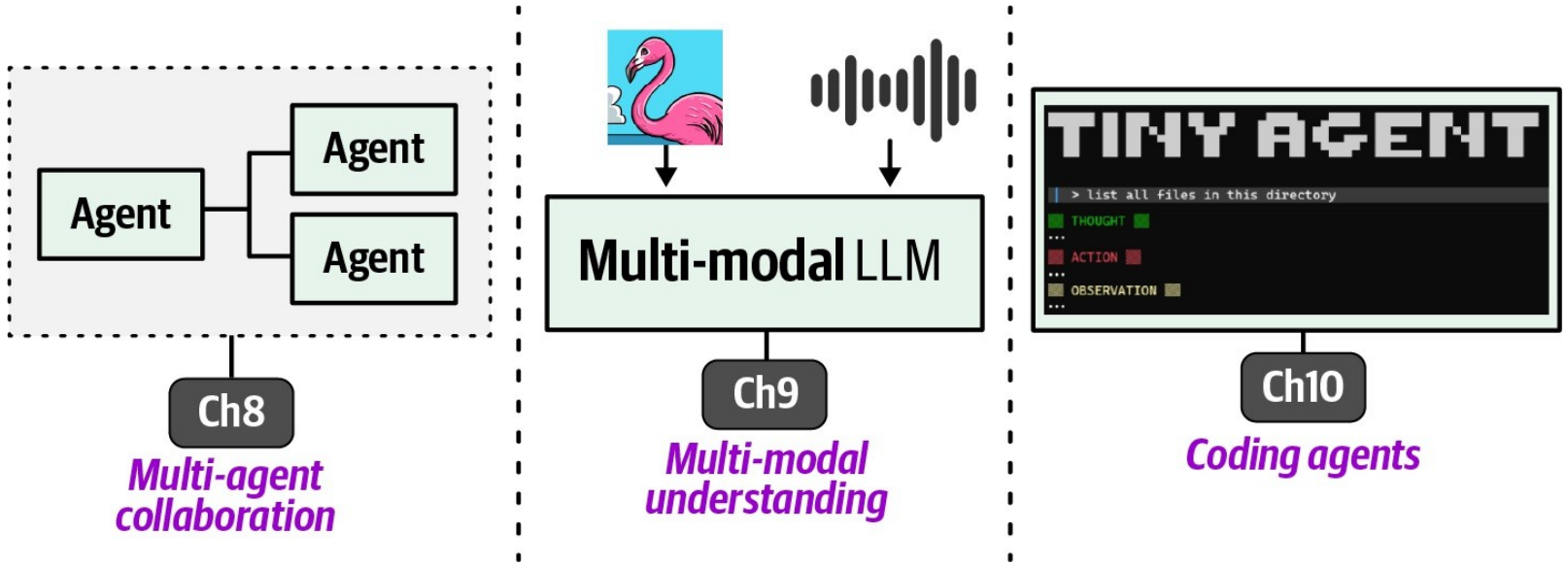
Ch 5 Tools and MCP

Ch 6 Planning and reflection

Ch 7 Evaluating agents

Figure 1-20 · An Illustrated Guide to AI Agents

Part II: specialized agents



Ch 8 · Multi-agent

Agents with different tasks consult each other

Ch 9 · Multi-modal

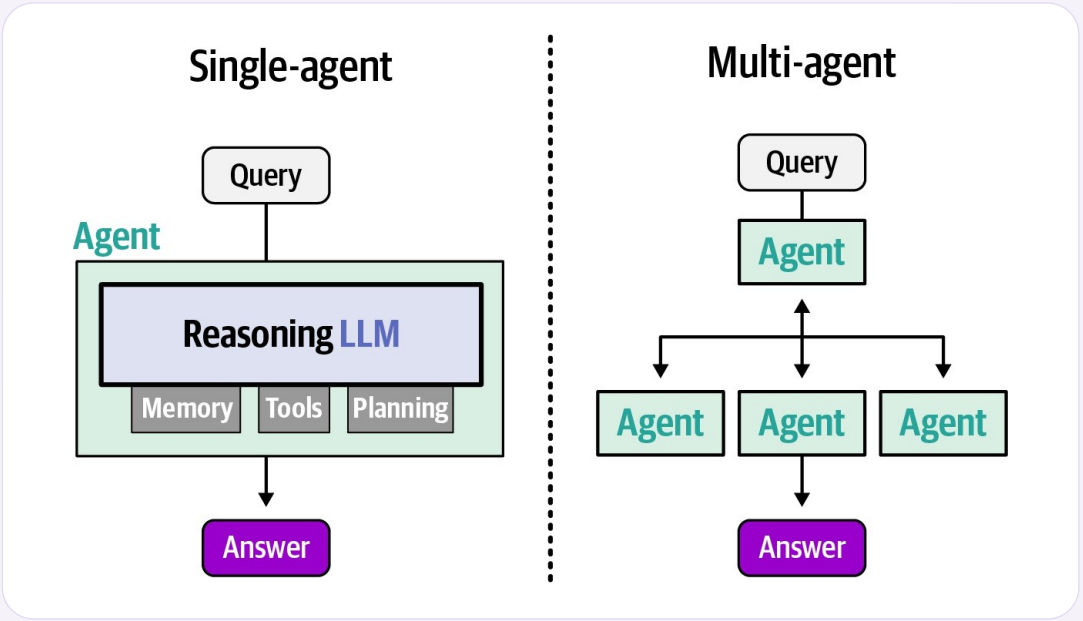
Seeing and hearing the world, not only text

Ch 10 · Coding agents

Run, test and fix code in a dedicated environment

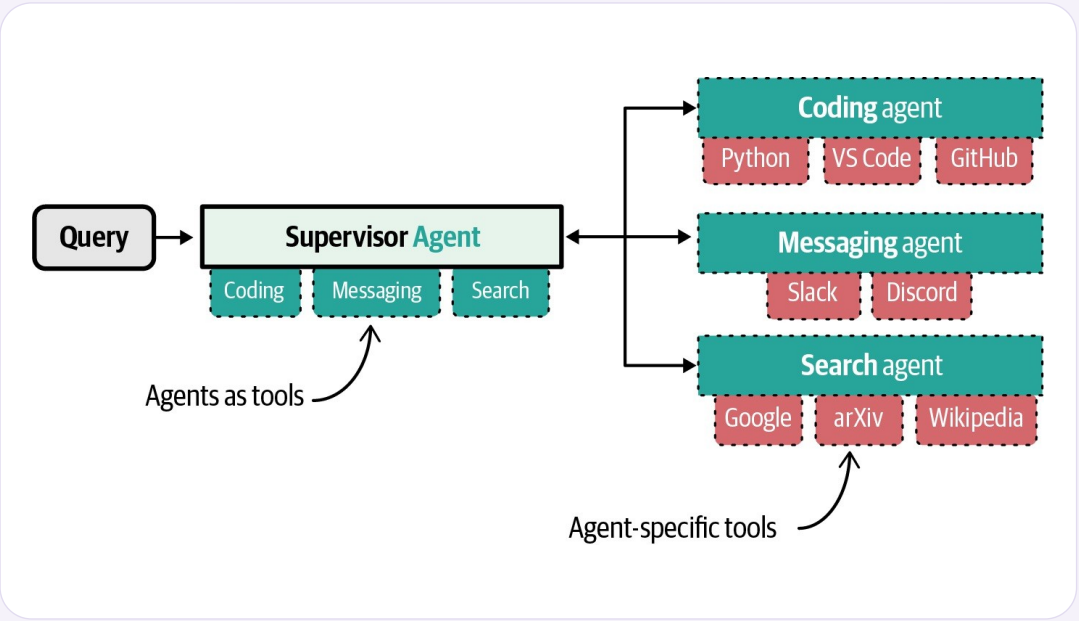
Figure 1-21 · An Illustrated Guide to AI Agents

Multi-agent collaboration



Many agents, specialized tasks

They interact and consult each other's specialties, and can beat a single agent.

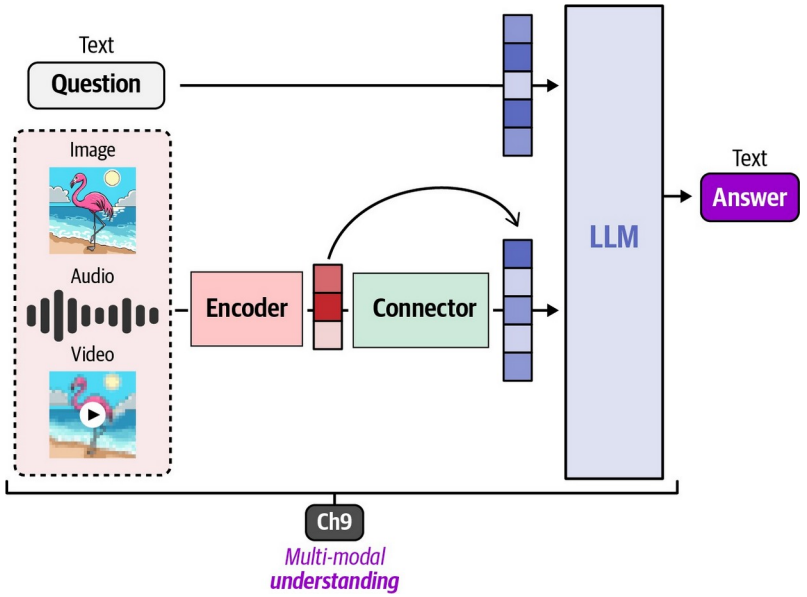


The supervisor pattern

A capable supervisor plans, decomposes and assigns tasks to agents with their own tools.

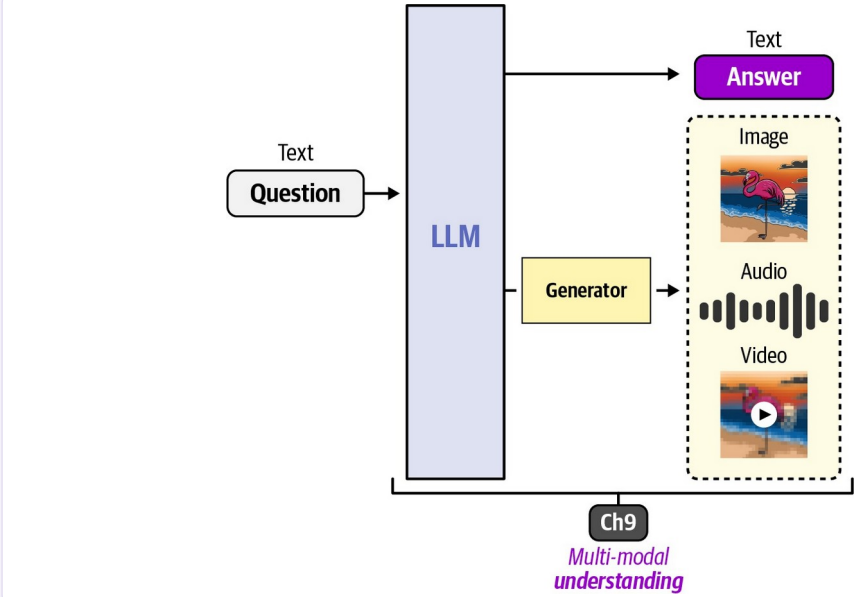
Figures 1-22 and 1-23 · An Illustrated Guide to AI Agents

Multi-modal agents: understand and generate



Understanding modalities

An encoder turns images, audio and video into numbers; a connector links them to the LLM.



Generating modalities

A generator produces output other than text, which is a very different process.

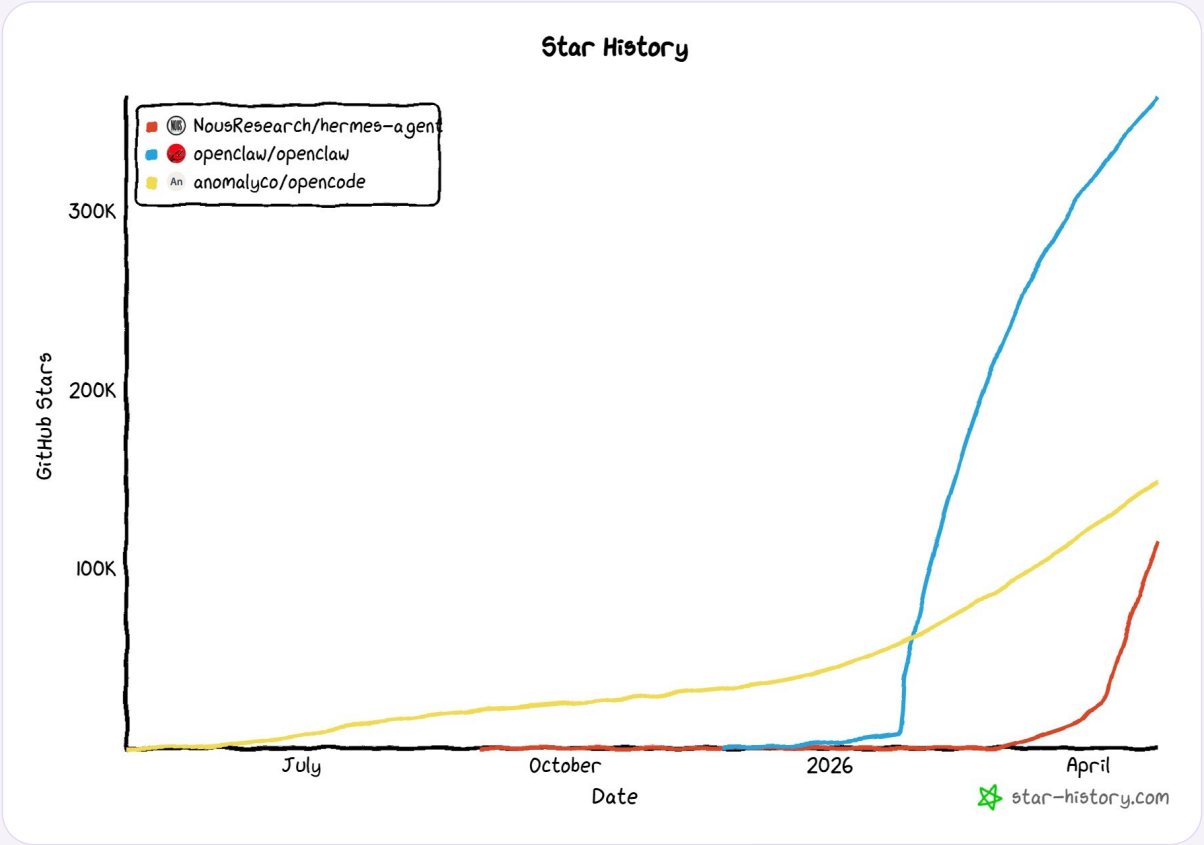
Figures 1-24 and 1-25 · An Illustrated Guide to AI Agents

Agent harnesses: the code around the LLM

Type	What it is	Examples
Terminal-based	Agents that run directly in the terminal	Claude Code, Gemini CLI, OpenAI Codex CLI
Code-based	Libraries you write code against to build your own agent	LangGraph, Smolagents, Pydantic AI
Personal assistant	Persistent agents that retain memory and skills across sessions	OpenClaw, Hermes Agent
Hosted	Agents that run in the cloud as a hosted product	Replit, v0, Manus
UI-based	Agents embedded inside a user-friendly interface	Antigravity, Cursor, Windsurf, GitHub Copilot

The list will age quickly, but every harness uses the same scaffolding: LLM, memory, tools and so on. Learn the foundations.

Personal-assistant harnesses take off



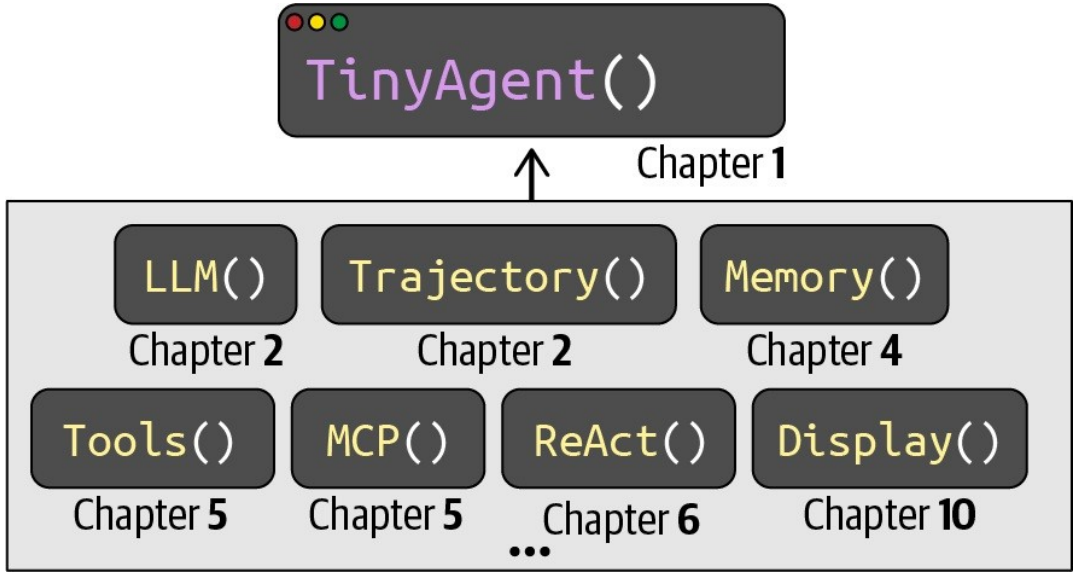
300,000

GitHub stars for OpenClaw within a couple of months of its release.

These harnesses are persistent and always on: chat with them via WhatsApp, Discord, Slack or email, and they solve tasks autonomously. They tend to get the most autonomy.

Figure 1-26 · An Illustrated Guide to AI Agents

Build an agent from scratch: TinyAgent



Learn by building. Each chapter adds one component to a small, modular Python agent.

Few dependencies. No packages that hide the complexity; everything is explained.

Code-based harness. Terminal implementation, with notebooks in the book's GitHub repository.

Figure 1-27 · An Illustrated Guide to AI Agents

The TinyAgent skeleton

```
class TinyAgent:
    def __init__(self):
        self.llm = None       # Ch 2 & 3
        self.memory = None    # Ch 4
        self.tools = None     # Ch 5
        self.planner = None   # Ch 6

    def run(self, task: str) -> str:
        return self._step(task)
```

Three methods: run, _step (one step, maybe a tool call) and _execute_action (run a tool).

```
agent.run("What is 2 + 2?")
'Received: What is 2 + 2?'
```

No brain yet: the skeleton only parrots the question until later chapters add the LLM, memory, tools and planning.

Key takeaways

1. An agent perceives its environment and acts on it; an LLM-backed agent uses a reasoning LLM as its brain.
2. Memory, tools, planning and reflection turn a regular LLM into an agent.
3. Autonomy is a spectrum: add guardrails and keep humans in the loop where stakes are high.
4. Evaluate the whole system: outcome, trajectory, reliability and safety.
5. Part II specializes the agent: multi-agent, multi-modal and coding agents.

Next: Chapters 2 and 3, the fundamentals of LLMs and reasoning LLMs